# The IITK Context-Aware Resume Diagnostic Engine

**Career Development Wing**

**150 Points**

## 1. Introduction & Problem Statement

Every placement season, students at IIT Kanpur invest immense effort into squeezing three years of rigorous academics, technical projects, and extracurricular leadership into a dense, single-page Student Placement Office (SPO) LaTeX template.

Because these standard tools fail us, students are left highly anxious. They rely on scattered, subjective, and often conflicting advice from seniors to figure out if their profile aligns with their target industry. There is currently no automated, reliable, and standardized way for a student to diagnose their resume's true strengths and weaknesses before day zero.

This problem statement challenges pools to build an **IITK-Specific Resume Diagnostic Engine**. You are not building a filtering tool for recruiters to reject candidates; you are building an intelligent, automated career advisor designed exclusively for the upliftment of IIT Kanpur students. The system must take a student's SPO-formatted PDF resume and a target industry track as inputs, flawlessly parse the complex formatting, semantically weigh the achievements against the specific target role, and output a customized gap analysis with highly actionable, hyper-specific feedback.

## 2. System Architecture & Core Modules

Your application must be structured around three core technical modules. Teams are free to choose their tech stack, provided the final product functions seamlessly.

### Module A: The LaTeX-PDF Parsing Engine

Standard text extraction tools often scramble multi-column LaTeX documents. Your parser must read text top-to-bottom, left-to-right, respecting the standard SPO multi-column grid. It must identify distinct sections accurately (e.g., Education, Experience, Projects) and successfully extract hyperlinks embedded within the PDF.

### Module B: The Semantic Weighting & NLP Engine

Keyword matching is insufficient. The engine must understand context:

- **Entity Recognition:** Recognize IITK jargon (e.g., "SURGE," "CPI," "AnC Council").
- **Impact Detection:** Detect whether bullet points contain quantifiable metrics (e.g., "reduced latency by 20%" vs. "worked on latency").
- **Action Verbs:** Evaluate the strength of verbs used to begin bullet points.

### Module C: The Advisory Dashboard

The interface must allow PDF uploads and track selection via a clean web app or CLI. It should display an overall "Profile Match Score" out of 100 for the selected role, alongside a structured report highlighting Top 3 Strengths, Critical Missing Elements, and Line-by-Line Formatting Fixes.

## 3. Role-Specific Evaluation Baselines

The engine must evaluate the same resume dynamically based on the target track. Your logic must account for the distinct expectations of at least 4 major roles:

- **Software Engineering (SDE):** Prioritizes DSA coursework, competitive programming (Codeforces), Open Source (GSoC), and full-stack projects. Penalizes missing GitHub links or generic projects lacking C++/Java/Python.

- **Quantitative Finance:** Heavily prioritizes exceptionally high CPI, rigorous mathematical coursework (Probability, Stochastic Calculus),and math/stat projects.Penalizes low CPIor lack of algorithmic proficiency.

- **Management Consulting:** Rewards ”spikes” in multiple areas (decent CPI + high-impact PoRs + sports/cultural). Leadership roles are crucial. Penalizes poor bullet formatting and lack of business impact metrics.

- **Core Engineering:** Prioritizes SURGE internships, core projects, research publications, and CAD/MATLAB proficiency. Penalizes generic web dev projects taking up space and missing core electives.

## 4. Deliverables

- **Diagnostic Engine Prototype:** A fully functional codebase hosted on a GitHub repository with a comprehensive README.md.

- **Architecture & Testing Report (ATR):** A detailed report (Max 10 pages) containing flowcharts of the parsing logic, explanation of semantic weighting, and testing results against mock resumes.

- **Presentation Deck (PD):** A high-level pitch deck (Max 12 slides) covering the architecture and a demonstration of handling edge cases.

## 5. Evaluation Scheme

| Parameter | Weight | Detailed Grading Criteria |
|---|---|---|
| **Diagnostic Accuracy** | 35% | Does the engine accurately identify strengths/weaknesses a human senior would spot? Is the Role-Targeted analysis functioning correctly when shifting tracks? |
| **Codebase & Arch.** | 25% | Quality of the GitHub repo, efficiency of PDF parsing (avoiding garbled text), and intelligent use of heuristics/NLP to weigh context. |
| **Actionability** | 25% | Specificity of advice. High scores awarded for pinpointing specific bullet points to fix (e.g., “Quantify the user base growth in Project 2”) rather than generic tips. |
| **UI/UX & Usability** | 15% | Is the dashboard or CLI intuitive? How well is the gap analysis visualized? |

## 6. Team Structure & Submission Guidelines

- **Team Composition:** 4 Y26s and 5 Y25 per pool for Engine Development and ATR. The Presentation must be delivered by 2 Y26 and 3 Y25.
- **Format:** The report and presentation deck should be in pdf format. The GitHub link must be on the title page of the report.
- **Naming Format:** PoolName_CDW_PS_DiagnosticEngine.pdf.
- **Late Penalties:** 5% deduction per 12 hours of delay past the deadline.